ASSIGNMENT 06 – MULTIMODAL SEARCH SYSTEM FOR E-COMMERCE

Information System Analysis and Design – three-layer design, UML models and a Python prototype

Student	KHANHBQ
ID	049
Class	04
Submitted files	Assignment06_KHANHBQ.vpp (Visual Paradigm project), assignment06/ (source code, products.json, sample images, README.md), demo_results/ (program output)

1. Introduction

Customers of an online shop want to find products and orders in whatever way is easiest for them: typing keywords, saying a sentence, or taking a photo of an item. This assignment designs and implements a small prototype of a **multimodal e-commerce search system**. Its central principle is: different inputs → common query representation → retrieval → ranking → results.

The report goes from the requirements, through the UML models built in Visual Paradigm (use case, three-layer architecture, sequence), to a Python program with the same structure as the models, and ends with an evaluation on a small test set.

2. Problem Description

The system contains products, customers and orders. Typical queries are “black running shoes”, “find Nike shoes under 100 dollars”, “show me products similar to this image”, “find my order 20261001” and “where is my latest order?”. Three input modes are supported:

Mode	Input	Processing in the prototype
Text	Keywords or a natural-language sentence	Normalisation, stop-word removal, price-filter extraction, keyword matching
Voice	Spoken sentence (audio)	**Simulated** speech-to-text → processed as text
Image	Product photograph	Image encoder f(I) → 128-d vector → cosine similarity

The prototype is not a production system: the data are 12 products and 4 orders, and the product images are synthetic drawings. The goal is to show how system design, UML modelling and an executable AI-oriented prototype fit together.

3. Requirements Analysis

Actor: Customer – a shopper who searches for products and looks up their own orders.

Functional requirements

ID	Functional requirement
FR1	Search products by keywords or a text sentence (Search by Keyword).

ID	Functional requirement
FR2	Search products by voice: the spoken sentence is converted to text and then searched as in FR1 (Search by Voice).
FR3	Search products similar to an uploaded image (Search by Image).
FR4	Combine text and an image in one multimodal query (weights λ_t, λ_i).
FR5	Filter by price when the query contains “under / below / less than N”.
FR6	Rank results by text match, image similarity and business information (stock), and show each product's score.
FR7	Look up an order by its ID (“find my order O001”) and the customer's latest order (“where is my latest order”).
FR8	View product / order details from the result list (View Product, View Order).

Non-functional requirements

- **Architecture:** three separate layers; the user interface never accesses the database directly (UI ↛ Database; UI → Application → Data).
- **Extensibility:** SpeechService and ImageService can be replaced by real models (Whisper, CLIP) without changing their callers; VectorIndex can be replaced by a vector database.
- **Performance:** on the sample data every query is answered in under one second.
- **Portability:** Python 3.9+, only numpy and pillow, runs offline.
- **Transparency:** results show the component scores (keyword, cosine) and the final score S(x|q).

Search inputs and outputs

- Inputs: a text string; a spoken sentence (simulated by its transcript); an image path; or text plus an image.
- Outputs: the top-5 products (name, price, stock, keyword score, cosine, final score S(x|q)); for order queries: order ID, status, total, date and customer.

4. Use Case Model

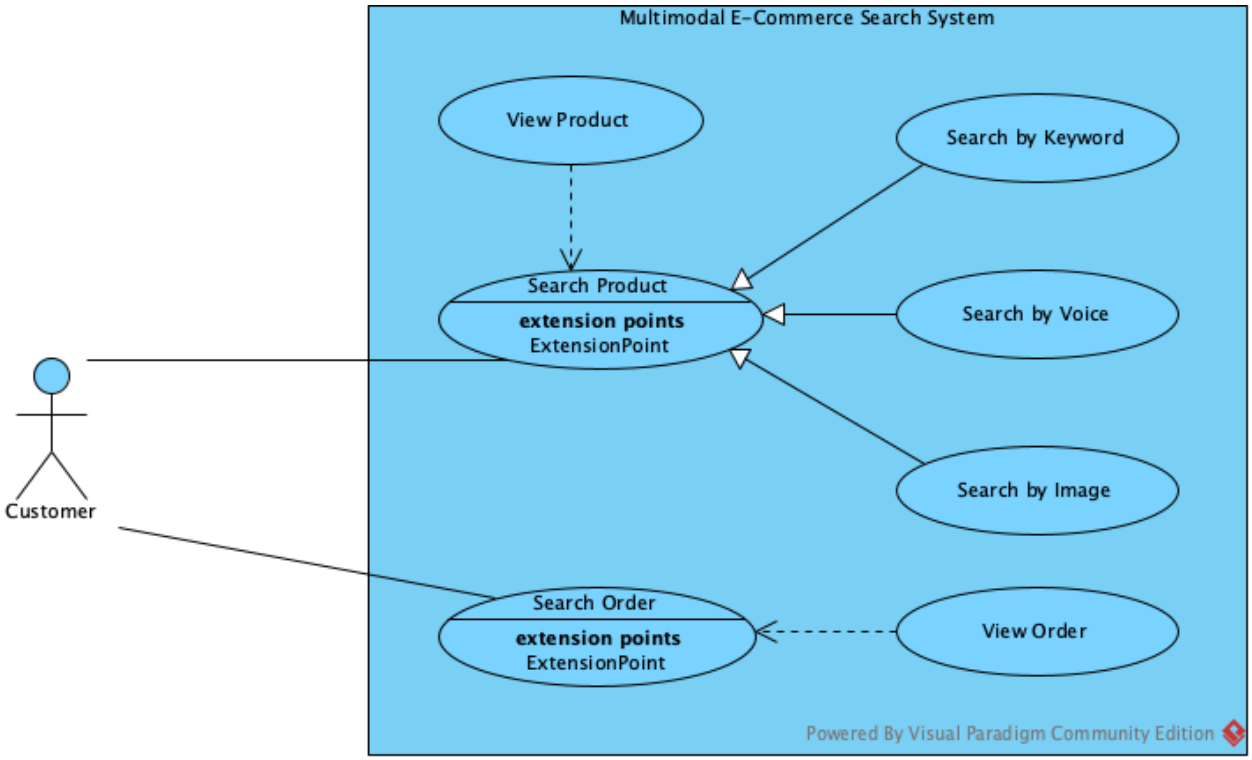


Figure 1. Use case diagram – diagram "UC01 Multimodal Search"

The Customer is associated with the two main use cases, *Search Product* and *Search Order*. The three ways of searching, *Search by Keyword / Voice / Image*, are specialisations (generalization) of Search Product: they share the goal and the "retrieve → rank → display" flow and differ only in how the input is read. *View Product* and *View Order* <<extend>> the corresponding search use case: the customer opens details only when they want to, after results are shown.

Use case	Short description	Realised by
Search Product	Find products for one common query	SearchUI, QueryService, SearchService, RankingService
Search by Keyword	Type keywords or a sentence	SearchUI.search_text
Search by Voice	Say the search sentence	VoiceInput, SpeechService, SearchUI.search_voice
Search by Image	Upload a photo	ImageUpload, ImageService, VectorIndex
View Product	Show details of a product in the results	SearchResultView
Search Order	Find an order by ID, or the latest order	SearchService.search_orders, OrderRepository
View Order	Show an order's status and total	SearchResultView.show_orders

5. Three-Layer Architecture

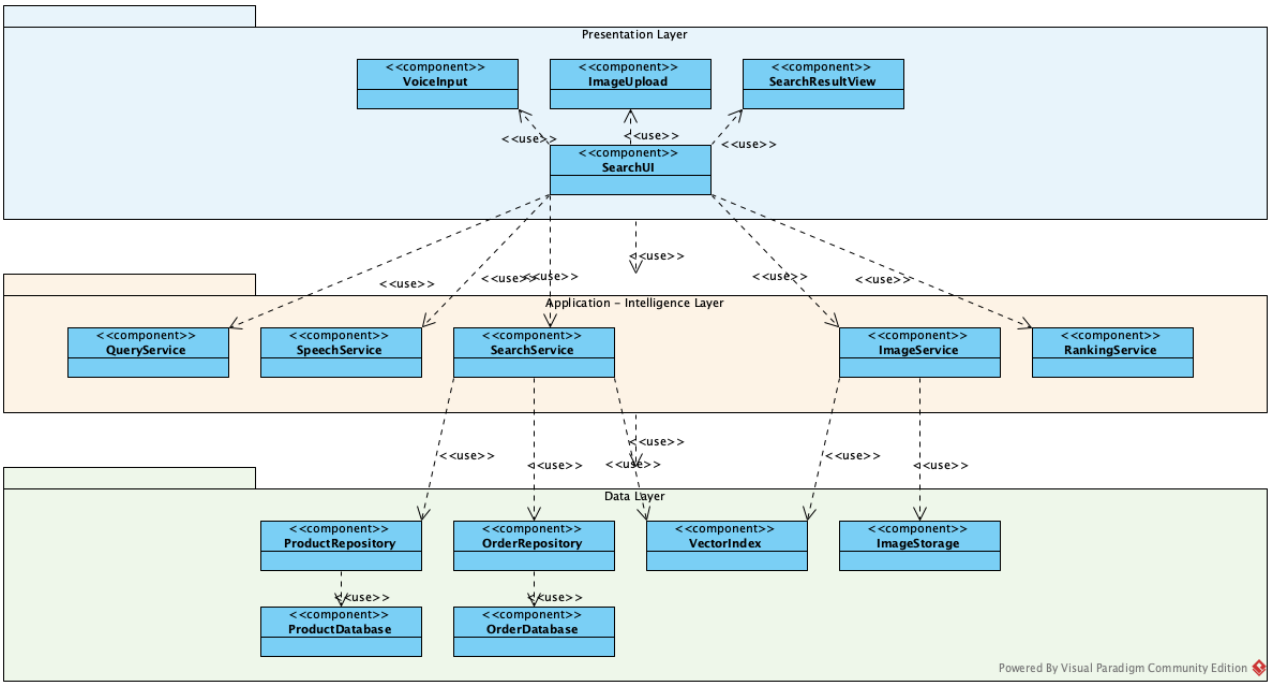


Figure 2. Three-layer component/package diagram – diagram "Three-Layer Architecture"

Each layer is a package; the components (<<component>>) depend on each other through <<use>> dependencies. Every arrow points downwards (Presentation → Application → Data) or stays inside one layer; there is **no** dependency from Presentation to Data. The VP model also has two package-level dependencies: Presentation Layer → Application – Intelligence Layer → Data Layer.

Layer	Responsibility	Components → Python files
Presentation	Collect the customer's input and display results	SearchUI, VoiceInput, ImageUpload, SearchResultView → presentation/search_ui.py
Application / Intelligence	Interpret queries, convert modalities, retrieve, filter and rank candidates	QueryService, SpeechService, ImageService, SearchService, RankingService → application/*.py
Data	Store products, orders, images and feature vectors	ProductRepository (+ products.json = ProductDatabase), OrderRepository (+ order list = OrderDatabase), VectorIndex, ImageStorage → data/*.py

6. Sequence Diagram – Voice Product Search

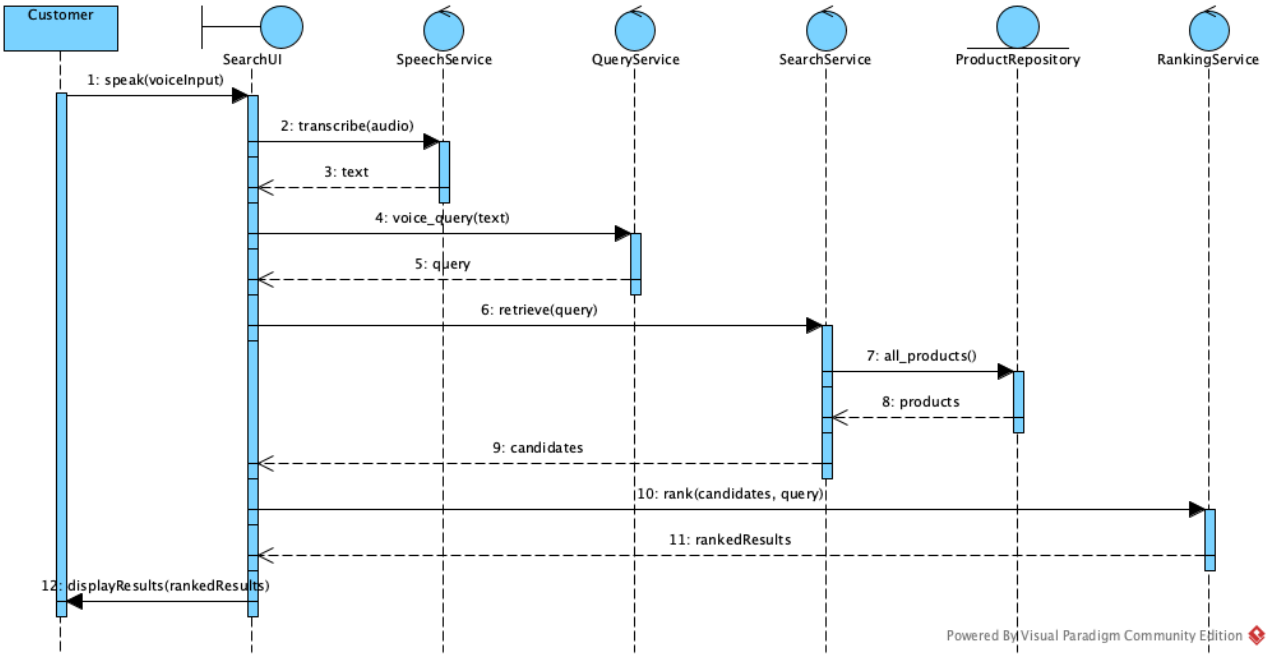


Figure 3. Sequence diagram – diagram "SD01 Voice Search"

The flow has 12 messages that cover the 8 steps given in the assignment:

- (1) The Customer speaks the query to SearchUI; (2–3) SearchUI sends the audio to SpeechService and gets the text back.
- (4–5) QueryService builds the common query object {type: voice, terms, filters}.
- (6–9) SearchService retrieves candidates; ProductRepository supplies the product data.
- (10–11) RankingService ranks the candidates; (12) SearchUI displays the results to the Customer.

SearchUI is the coordinating boundary, the services are controls and ProductRepository is the data-access entity. SearchUI never calls ProductRepository directly, as the layering requires. The call order is exactly that of `SearchUI.search_voice` in the source code.

Note on the figures: the three diagrams are exported directly from Assignment06_KHANHBQ.vpp with Visual Paradigm Community Edition (the small "Powered By Visual Paradigm Community Edition" text in the corner is CE's standard attribution).

7. Python Implementation

The directory structure follows the architecture: every component in the diagrams is a Python class.

```
assignment06/
  main.py              wiring of the layers + demo / command line
  evaluate.py          test set, success rate
  presentation/search_ui.py   SearchUI, VoiceInput, ImageUpload, SearchResultView
  application/query_service.py  speech_service.py  image_service.py
              search_service.py  ranking_service.py
  data/product_repository.py  order_repository.py  vector_index.py  image_storage.py
       products.json (12 products)   images/   query_images/
  tools/make_images.py  sample-image generator
```

Common query object (QueryService). Every method returns a dict; a sentence that contains "order" becomes an order query, and a phrase such as "under N" becomes a price filter:

```
    def _parse(self, text, mode):
        raw = text.strip()
        lowered = raw.lower()

        if re.search(r"\border\b", lowered):
            match = ORDER_ID_RE.search(lowered)
            return {
                "type": "order",
                "source": mode,
                "query": raw,
                "order_id": match.group(1).upper() if match else None,
                "latest": bool(re.search(r"\b(latest|last|recent)\b", lowered)),
            }

        filters = {}
        price = PRICE_RE.search(lowered)
        if price:
            filters["max_price"] = float(price.group(1))
            lowered = PRICE_RE.sub(" ", lowered)

        words = re.findall(r"[a-z0-9\-]+", lowered)
        terms = [w for w in words if w not in STOPWORDS]

        return {
            "type": mode,
            "query": raw,
            "terms": terms,
            "filters": filters,
        }
```

Retrieval (SearchService). It keeps the keyword-matching algorithm of the assignment (score = number of query words found in name + category + colour + brand), adds token matching with a simple plural rule and the price filter, and takes the top-k from VectorIndex for image queries. `retrieve` merges both sources into the candidate set C(q), still unranked:

```
    def retrieve(self, query):
        """Candidate set C(q) for a common query object: {id: {product, text, image}}."""
        candidates = {}

        def add(product, signal, value):
            entry = candidates.setdefault(product["id"],
                                          {"product": product, "text": 0, "image": 0.0})
            entry[signal] = value

        if query.get("terms"):
            for product, score in self.search_text(query["terms"], query.get("filters")):
                add(product, "text", score)

        if query.get("embedding") is not None:
```

```
            for product, score in self.search_image(query["embedding"]):
                if product["price"] <= query.get("filters", {}).get("max_price", float("inf")):
                    add(product, "image", score)

        return list(candidates.values())
```

Voice (SpeechService) – simulated. Speech recognition is **not** actually performed: the `audio_input` argument already is the sentence a speech-to-text engine would return, and VoiceInput only simulates recording. `transcribe` can be replaced by Whisper without changing any other class.

Ranking (RankingService) is separate from retrieval and computes the final score of every candidate:

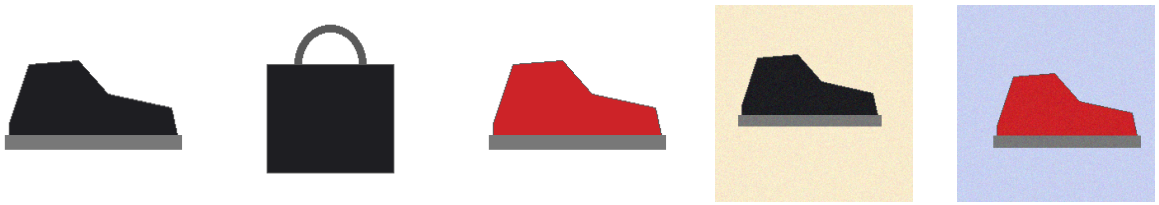
```
    def rank(self, candidates, query, k=5):
        alpha, beta, gamma = self.weights(query)
        n_terms = max(len(query.get("terms") or []), 1)

        ranked = []
        for c in candidates:
            s_text = c["text"] / n_terms
            s_image = max(c["image"], 0.0)
            s_business = self.business_score(c["product"])
            score = alpha * s_text + beta * s_image + gamma * s_business
            ranked.append({**c, "s_text": s_text, "s_image": s_image,
                           "s_business": s_business, "score": round(score, 4)})

        ranked.sort(key=lambda r: (r["score"], r["text"], r["image"]), reverse=True)
        return ranked[:k]
```

8. Multimodal Search Method

Image representation. The encoder $f_\theta(I)$ is hand-made (no neural network) so that the prototype runs offline. The image is resized to 64×64, the background colour is estimated from the image border, and pixels that differ from it form the object. The vector consists of (a) a 4×4×4 RGB colour histogram of the object (64 values) and (b) an 8×8 occupancy grid describing its shape (64 values); each part is L2-normalised and the two are concatenated (128-d). A photo on a different background, shifted or noisy, therefore still matches the catalogue image.



p01 (catalogue) *p03 (catalogue)* *p12 (catalogue)* *query (black)* *query (red)*

Figure 4. Catalogue product images and query photos (different background, position/size and noise)

Similarity is the cosine similarity, and retrieval takes the top-k:

$$s(I_q, I_p) = z_q^T z_p / (\|z_q\| \|z_p\|), \; R(q) = TopK_{x \in X} s(q, x)$$

Ranking. The final score combines three signals:

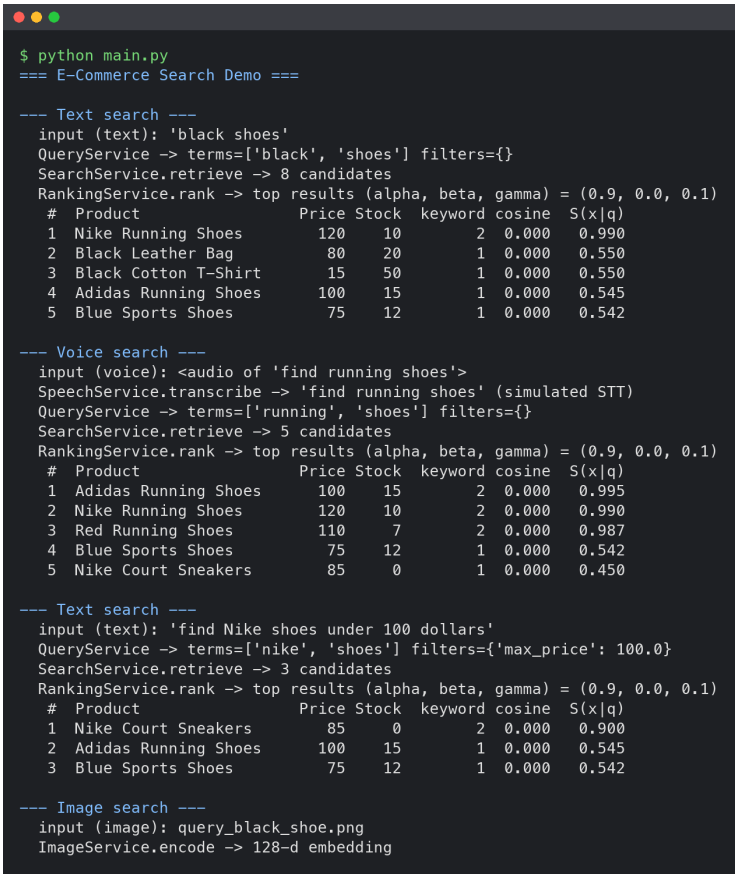
$$S(x|q) = \alpha \cdot S_{text} + \beta \cdot S_{image} + \gamma \cdot S_{business}$$

- S_{text} = matched words / number of query words (0..1); S_{image} = cosine (0..1);
- $S_{business}$ = 0.8·[in stock] + 0.2·min(stock, 20)/20 – favours products that are in stock (stock-aware ranking);
- text/voice: (α, β, γ) = (0.9, 0, 0.1); image: (0, 0.9, 0.1); multimodal: $\alpha = 0.9 \cdot \lambda_t$, $\beta = 0.9 \cdot \lambda_i$, $\gamma = 0.1$ with $\lambda_t + \lambda_i = 1$ (default 0.5 / 0.5).

A multimodal query merges the candidates of both sources and computes $S(p|q) = \lambda_t S_t(p|q_t) + \lambda_i S_i(p|q_i)$ (plus the business term). For example, with “running shoes” + a photo of a red shoe, all three running shoes match 2/2 keywords, and the image lifts Red Running Shoes to the top (cosine 0.959) – something neither modality achieves alone.

9. Experimental Results

`python main.py` runs the three required queries (text, voice, image) plus a price-filtered query, a text + image query and two order queries. Each query prints its input, the processing steps, the returned products and their ranking scores.



```
$ python main.py
=== E-Commerce Search Demo ===

--- Text search ---
  input (text): 'black shoes'
  QueryService -> terms=['black', 'shoes'] filters={}
  SearchService.retrieve -> 8 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.9, 0.0, 0.1)
   #  Product                Price Stock  keyword cosine  S(x|q)
   1  Nike Running Shoes       120    10        2  0.000   0.990
   2  Black Leather Bag         80    20        1  0.000   0.550
   3  Black Cotton T-Shirt      15    50        1  0.000   0.550
   4  Adidas Running Shoes     100    15        1  0.000   0.545
   5  Blue Sports Shoes         75    12        1  0.000   0.542

--- Voice search ---
  input (voice): <audio of 'find running shoes'>
  SpeechService.transcribe -> 'find running shoes' (simulated STT)
  QueryService -> terms=['running', 'shoes'] filters={}
  SearchService.retrieve -> 5 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.9, 0.0, 0.1)
   #  Product                Price Stock  keyword cosine  S(x|q)
   1  Adidas Running Shoes     100    15        2  0.000   0.995
   2  Nike Running Shoes       120    10        2  0.000   0.990
   3  Red Running Shoes        110     7        2  0.000   0.987
   4  Blue Sports Shoes         75    12        1  0.000   0.542
   5  Nike Court Sneakers       85     0        1  0.000   0.450

--- Text search ---
  input (text): 'find Nike shoes under 100 dollars'
  QueryService -> terms=['nike', 'shoes'] filters={'max_price': 100.0}
  SearchService.retrieve -> 3 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.9, 0.0, 0.1)
   #  Product                Price Stock  keyword cosine  S(x|q)
   1  Nike Court Sneakers       85     0        2  0.000   0.900
   2  Adidas Running Shoes     100    15        1  0.000   0.545
   3  Blue Sports Shoes         75    12        1  0.000   0.542

--- Image search ---
  input (image): query_black_shoe.png
  ImageService.encode -> 128-d embedding
```

Figure 5. Output of main.py (part 1): text, voice, text with a price filter

```
$ python main.py  (continued)
  SearchService.retrieve -> 5 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.0, 0.9, 0.1)
   #  Product                Price Stock  keyword cosine  S(x|q)
   1  Nike Running Shoes       120    10        0  0.896   0.896
   2  Black Leather Bag         80    20        0  0.847   0.862
   3  Black Cotton T-Shirt      15    50        0  0.774   0.796
   4  Black Smart Watch        150     6        0  0.784   0.792
   5  Blue Sports Shoes         75    12        0  0.436   0.484

--- Multimodal search ---
  input (text + image): 'running shoes' + query_red_shoe.png
  lambda_t = 0.5, lambda_i = 0.5
  QueryService -> terms=['running', 'shoes'] filters={}
  SearchService.retrieve -> 6 candidates
  RankingService.rank -> top results (alpha, beta, gamma) = (0.45, 0.45, 0.1)
   #  Product                Price Stock  keyword cosine  S(x|q)
   1  Red Running Shoes        110     7        2  0.959   0.969
   2  Nike Running Shoes       120    10        2  0.504   0.767
   3  Adidas Running Shoes     100    15        2  0.413   0.731
   4  Blue Sports Shoes         75    12        1  0.495   0.540
   5  Red Canvas Backpack       60     8        0  0.808   0.452

--- Order search ---
  input (text): 'find my order O001'
  QueryService -> order query (order_id=O001, latest=False)
  Order: O001
  Status: Shipped
  Total: 150
  Date: 2026-09-20  Customer: C001

--- Order search ---
  input (voice): <audio of 'where is my latest order'>
  SpeechService.transcribe -> 'where is my latest order' (simulated STT)
  QueryService -> order query (order_id=None, latest=True)
  Order: 20261001
  Status: Pending
  Total: 25
  Date: 2026-10-01  Customer: C001
```

Figure 6. Output of main.py (part 2): image, text + image, order search

Observations: “black shoes” puts Nike Running Shoes (both words match) first, as in the assignment's example. For “find Nike shoes under 100 dollars” the price filter removes Nike Running Shoes ($120), so Nike Court Sneakers ($85) comes first; it is out of stock, so its score is 0.900 instead of 0.990. The black-shoe photo returns Nike Running Shoes with cosine 0.896.

Evaluation on the test set (`python evaluate.py`): 20 queries – 6 text, 3 voice, 7 image, 2 multimodal and 2 order queries. A query counts as successful when the top-1 result is the expected product or order.

```
$ python evaluate.py
 #  Mode       Query                                Top-1 result           Score  Success
 1  text       black shoes                          Nike Running Shoes     0.990  Yes
 2  text       white polo shirt                     White Polo Shirt       0.989  Yes
 3  text       find Nike shoes under 100 dollars    Nike Court Sneakers    0.900  Yes
 4  text       red bag                              Red Canvas Backpack    0.988  Yes
 5  text       cheap watch under 60                 Silver Analog Watch    0.544  Yes
 6  text       leather handbag                      Black Leather Bag      0.550  Yes
 7  voice      find running shoes                   Adidas Running Shoes   0.995  Yes
 8  voice      show me a blue cap                   Blue Denim Cap         1.000  Yes
 9  voice      i want a smart watch                 Black Smart Watch      0.986  Yes
10  image      query_black_shoe.png                 Nike Running Shoes     0.896  Yes
11  image      query_red_shoe.png                   Red Running Shoes      0.950  Yes
12  image      query_white_shoe.png                 Adidas Running Shoes   0.786  Yes
13  image      query_black_bag.png                  Black Leather Bag      0.962  Yes
14  image      query_blue_cap.png                   Blue Denim Cap         0.955  Yes
15  image      query_silver_watch.png               Silver Analog Watch    0.969  Yes
16  image      query_black_tshirt.png               Black Cotton T-Shirt   0.961  Yes
17  multimodal running shoes + query_red_shoe.png   Red Running Shoes      0.969  Yes
18  multimodal nike + query_white_shoe.png          Nike Running Shoes     0.805  No (expected Nike Court Sneakers)
19  text       find my order O001                   O001                      -  Yes
20  voice      where is my latest order             20261001                  -  Yes

Total queries: 20   Successful: 19   Success rate: 95.0%
```

Figure 7. Output of evaluate.py

Metric	Value
Number of test queries	20
Number of successful queries	19
Success rate = 19 / 20	95.0 %
Per mode	text 6/6, voice 3/3, image 7/7, multimodal 1/2, order 2/2

Incorrect result (#18). The query "nike" + a photo of a white shoe expects Nike Court Sneakers (a white Nike shoe), but the system returns Nike Running Shoes. Nike Court Sneakers has the more similar image (cosine 0.768 vs 0.589), but it is out of stock (stock = 0), so it loses 0.09 business points and scores only 0.795, behind Nike Running Shoes (0.805), which also matches "nike". This is a deliberate trade-off of stock-aware ranking, but it shows that γ has to be tuned on real data. Also, in #12 Adidas Running Shoes and Nike Court Sneakers have identical pictures (same colour and shape), so their cosine scores are equal – the simple encoder cannot tell brands apart.

10. Discussion

- **The architecture makes extension easy:** because every modality becomes the same query object, adding multimodal search needed only one new method in QueryService and one weight set in RankingService; SearchService did not change.
- **Retrieval ≠ ranking:** separating the two steps lets the ranking policy change (e.g. favouring in-stock items) without touching how candidates are retrieved.
- **Limitations:** speech-to-text is simulated; the image encoder captures only colour and shape; keyword matching knows no synonyms ("sneakers" ≠ "shoes") and no typo tolerance; the data set is small and the images are synthetic, so 95 % does not reflect real-world accuracy.
- **Future work:** Whisper for voice; CLIP for both images and text (one shared vector space, semantic search); FAISS or a vector database instead of VectorIndex; a web interface; filters by product category and order status.

11. Conclusion

This work analysed the requirements (8 functional, 5 non-functional), built three UML diagrams in Visual Paradigm and implemented a three-layer Python prototype with text, simulated voice, image, text + image and order search, reaching 19/20 correct queries on the test set. The main lesson is that an AI-enabled e-commerce system is not only an AI model: it is a complete software system following User Interface → AI/Application Logic → Data, in which every kind of input is turned into one unified search process.

Project repository: https://github.com/qkhanhbe/Multimodal-Search-System-for-E-Commerce